

Puzzle Problem

Name : Asmit Sahu Roll : 23052231

```
In [1]: import heapq

GOAL_STATE = (1, 2, 3,
              4, 5, 6,
              7, 8, 0)

MOVES = {
    0: [1, 3],
    1: [0, 2, 4],
    2: [1, 5],
    3: [0, 4, 6],
    4: [1, 3, 5, 7],
    5: [2, 4, 8],
    6: [3, 7],
    7: [4, 6, 8],
    8: [5, 7]
}

# ----- Heuristics -----

def h1_misplaced(state):
    """Number of misplaced tiles"""
    return sum(1 for i in range(9) if state[i] != 0 and state[i] != GOAL_STATE[i])

def h2_manhattan(state):
    """Sum of Manhattan distances"""
    distance = 0
    for i in range(9):
        if state[i] != 0:
            goal_index = GOAL_STATE.index(state[i])
            x1, y1 = divmod(i, 3)
            x2, y2 = divmod(goal_index, 3)
            distance += abs(x1 - x2) + abs(y1 - y2)
    return distance

# ----- A* Algorithm -----

def astar(start, heuristic):
    pq = []
    heapq.heappush(pq, (heuristic(start), 0, start, None))
    visited = {}
    nodes_explored = 0

    while pq:
        f, g, state, parent = heapq.heappop(pq)

        if state in visited:
            continue
```

```

        visited[state] = parent
        nodes_explored += 1

    if state == GOAL_STATE:
        return g, nodes_explored

    zero = state.index(0)
    for move in MOVES[zero]:
        new_state = list(state)
        new_state[zero], new_state[move] = new_state[move], new_state[zero]
        new_state = tuple(new_state)

        if new_state not in visited:
            heapq.heappush(
                pq,
                (g + 1 + heuristic(new_state), g + 1, new_state, state)
            )

    return None, nodes_explored

# ----- Test -----

if __name__ == "__main__":
    start_state = (1, 2, 3,
                   4, 0, 6,
                   7, 5, 8)

    depth_h1, nodes_h1 = astar(start_state, h1_misplaced)
    depth_h2, nodes_h2 = astar(start_state, h2_manhattan)

    print("Using H1 (Misplaced Tiles):")
    print("Solution Depth:", depth_h1)
    print("Nodes Explored:", nodes_h1)

    print("\nUsing H2 (Manhattan Distance):")
    print("Solution Depth:", depth_h2)
    print("Nodes Explored:", nodes_h2)

```

Using H1 (Misplaced Tiles):

Solution Depth: 2

Nodes Explored: 3

Using H2 (Manhattan Distance):

Solution Depth: 2

Nodes Explored: 3

In []: